

CARTOGRAPHIE & PARCOURS COMPLET DU MODÈLE BPMN 2.0 (XML)

Visualisation de bout en bout de l'ingestion, persistance en base de données et restitution réactive IHM

⚡ Flowable Engine 7.0.1

🐘 PostgreSQL 4 Schémas

⚙️ React Kanban & Flow

1 Modélisation & Export

Flowable UI Modeler (Web)

1.1 Conception Visuelle

Port 8082

L'analyste dessine le workflow des incidents :

- Événement départ : StartEvent
- Tâches humaines : UserTask
- Flèches orientées : SequenceFlow

1.2 Document XML BPMN 2.0

OMG Standard

```
<!-- Fichier généré -->
<process id="incidentWorkflow">
  <userTask id="taskNouveau"
    name="Nouveau" />
  <userTask id="taskAssigne"
    name="Assigné" />
  <userTask id="taskEnCours"
    name="En cours" />
  <userTask id="taskResolu"
    name="Résolu" />
  <sequenceFlow sourceRef="taskEnCours"
    targetRef="taskResolu" />
</process>
```

💡 **Avantage** : Aucune logique de transition n'est codée en dur dans le code Java. Le dessin XML devient le contrat d'exécution.

2 Ingestion & Synchronisation

Spring Boot Java Backend

2.1 Déploiement Hot-Reload

WorkflowController

```
// Réception du fichier .bpmn20.xml
@PostMapping("/deploy-flowable")
public Response deploy(MultipartFile file) {
    repositoryService.createDeployment()
        .addInputStream(resName, file.getInputStream())
        .deploy(); // Parse & persiste

    // Déclenche la synchronisation réactive
    workflowService.syncWithFlowable();
}
```

2.2 Algorithme de Sync

WorkflowService

```
// Lecture de l'arbre BPMN en mémoire
BpmnModel model = repo.getBpmnModel(id);
List<UserTask> tasks = model.findTasks();

// Met à jour incidentflow.workflow_states
workflow.getStates().clear();
for (String name : orderedNames) {
    workflow.getStates().add(name);
}
wfRepo.save(workflow);
cacheManager.clearCache("workflows");
```

🔄 **Réactivité** : Normalise automatiquement le fichier vers .bpmn20.xml pour éviter le piège des parenthèses (2) de Chrome.

3 Persistance Multi-Schémas

PostgreSQL (IncidentFlow_db)

3.1 Schéma "flowable"

47 Tables Isolées

Table PostgreSQL	Ce qui y est stocké
act_ge_bytearray	Le fichier XML binaire (BLOB dans bytes_)
act_re_procdef	Métadonnées parsées : key_, version_=1,2..
act_re_deployment	Horodatage du déploiement (deploy_time_)
act_ru_task	Tâches actives en cours d'exécution
act_hi_taskinst	Journal d'audit immuable et durée SLA

3.2 Schéma "incidentflow"

11 Tables Métier

Table Métier	Impact après import XML
workflow_states	Nouveaux statuts extraits : Nouveau, Résolu...
incidents	Liaison Soft FK via process_instance_id

🛡️ **Sécurité** : Aucune contrainte SQL dure entre les schémas. Zéro couplage, purges d'audit sans risque de deadlock.

4 Rendu Client Réactif

React SPA (Port 3001)

4.1 Tableau Kanban

KanbanView.jsx

Génère dynamiquement les colonnes selon la liste ordonnée retournée par l'API :

🟢 Nouveau 🟡 Assigné 🟠 En cours 🟤 Résolu

4.2 Stepper Horizontal

IncidentDetailView

Affiche la barre de progression ordonnée sur chaque ticket :

(1) ✓ → (2) ✓ → (3) ● → (4) ○ → (5)

4.3 Schéma Animé

React Flow Live

Diagramme interactif avec zoom, pan, minimap et flux de particules animées en continu sur les transitions actives.

🌟 **Zero Hardcode** : L'interface s'adapte instantanément dès qu'un état est renommé dans le XML.



Le Cycle de Vie Runtime : Ce qui se passe quand un incident réel est créé et résolu

Exécution déterministe assurée par le moteur Flowable Runtime lors des interactions utilisateurs

1. Déclaration Ticket

runtimeService.startProcess()
Génère process_instance_id



2. Tâche Active

Insertion dans act_ru_task
Statut =
currentTask.getName()



3. Transition Kanban

taskService.complete(taskId)
Le jeton avance selon le XML



4. Traçabilité Finale

Archivage dans
act_hi_taskinst
Audit ISO 27001 infalsifiable